AgentOps for Real

Evals, Tracing, Regression Tests, and Control for AI Agents

Taming Wild AI Agents Before They Eat Your Production Environment

Brian Haydin
Solution Architect, Concurrency
Milwaukee Global AI Community Organizer

Let's connect on LinkedIn

linkedin.com/in/brianhaydin

Why This Matters Now

The Awkward Teenage Phase of Agent Adoption

~50% Projects stuck in POC/Pilot

51% Cite monitoring as top barrier

>40% May be canceled by 2027

33% Enterprise software to include agents by 2028

Adoption Pressure **Production Friction**

v

s.

Poll: How many agent POCs have quietly wandered into the woods?

Sources: Gartner, Industry Surveys 2025-2026,
Dynatrace's 2026 report: https://www.dynatrace.com/news/press-release/pulse-of-agentic-ai-2026

The Great Tension

Adoption Hype | **Production Fear**

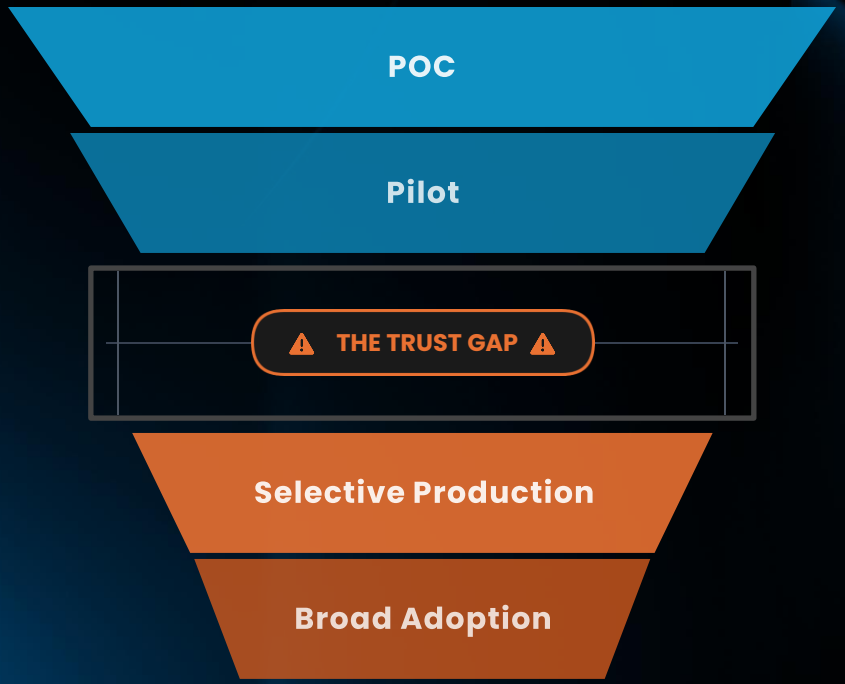
"Deploy it now!" | "It's hallucinating!"

Quick Poll:

"Raise your hand if you've seen a promising POC quietly wander into the woods and never come back."

From Demo to Deployment

The POC Trap



? Why do we stall here?

Will it behave consistently?
Unlike software, 'correct' is subjective.

What happens on updates?
Model version 3.5 → 4.0 breaks prompts.

How do we catch drift?
Silent failures in reasoning logic.

The Solution
AgentOps closes the trust gap. →

Concurrency

Friday at 4:37 PM

When a 'harmless improvement' goes sideways

- **4:37 PM** — Deploy
- **5:05 PM** — First subtle misroute
- **5:20 PM** — Clustering goes sideways
- **5:28 PM** — Human-review mode

Infrastructure was green. Behavior was wrong. That's what makes agents different.

Why Traditional Testing Fails

Concurrency

Software 1.0

Deterministic Failure

"Fails like a Machine"

- Hard crashes & Exceptions
- Null pointers & Type errors
- Broken API contracts (400/500)
- Repeatable, binary outcomes

Solved by: Unit Tests, Assertions

vs

Agentic AI

Probabilistic Failure

"Fails like a Junior Employee"

- Behavioral Drift & Silent Regression
- Wrong Tool Choice
- Polished but factually wrong answers
- Compound multi-step logic errors

Solved by: Evals, Tracing, Human Review

The Conclusion: **You need Evals and Traces, not just asserts.**

What is AgentOps?

Observe + Evaluate + Gate + Control

OBSERVE

- Traces
- Inputs/Outputs
- Tool calls
- Latency
- Tokens

GATE

- Quality thresholds
- Pass/fail criteria
- CI/CD integration

EVALUATE

- Golden prompts
- Datasets
- Scoring
- Regression checks

CONTROL

- Review modes
- Permissions
- Kill switches
- Shutdown

2026 AgentOps Trends That Matter

What's actually sticking vs. what's hype

OpenTelemetry as plumbing

Trace-driven evals

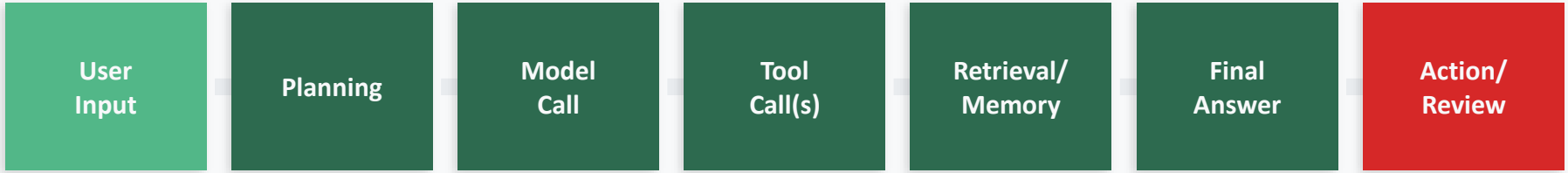
Production → eval datasets

Hybrid scoring

CI/CD quality gates

Anatomy of a Tool-Using Agent Run

Not one model call — a workflow wearing a chatbot costume



Every step is a potential failure point. Traces show you where.

Anatomy of an Eval

A good eval checks behavior. A great eval checks behavior, risk, and repeatability.

Every eval needs these parts:

1	**Scenario**	What user situation are we testing?
2	**Input**	Prompt, context, tool state
3	**Expected Behavior**	Tool choice, safety, escalation, output shape
4	**Assertion Type**	Rule-based / Semantic / LLM judge / Human
5	**Pass Threshold**	100% / 0.85 / pass-fail / max latency/cost
6	**Risk Level**	Low / Medium / High impact to business

Good vs Great:

Good Eval

- Real scenario
- Clear expected behavior
- Right score type
- Pass threshold

Great Eval

- Uses real production failures
- Checks intermediate steps
- Versioned & repeatable
- Runs in CI/CD
- Ties to business risk

Bad eval: "Did the answer sound smart?" Great eval: "Did the system behave correctly, safely, and consistently?"

Evals That Are Actually Useful

Your eval set is a contract, not a vibes folder

Prompt	Expected Behavior	Score Type	Pass Threshold
Route billing question	Tool: billing_search, no escalation	Rule-based	100%
Explain refund policy	Semantic match to policy doc	Similarity	>0.85
Unsafe request	Rejection + safe response	LLM Judge	Pass/Fail

Need: rule-based + semantic similarity + LLM judges. No single metric tells the whole story.

DEMO 1

Regression Harness in .NET

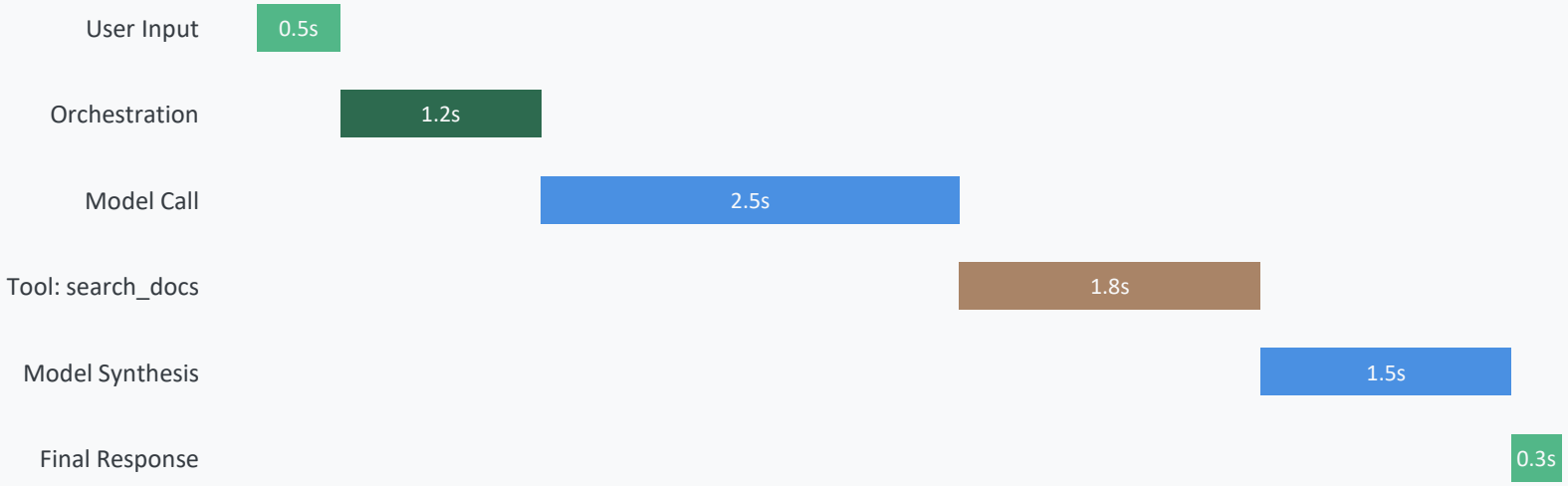
Change → Run Tests → Fail Early → Fix Safely

Making agents testable like real software

Tracing and Replay

An eval tells you WHAT. A trace tells you WHERE.

Trace Waterfall Example:



Traces show: tool calls, retries, latency, token usage. Replay to debug drift.

Intermediate Reasoning Steps: What to Capture

Log decision breadcrumbs, not brain soup.

Step	What to Capture	Why It Matters
Planning / Routing	• Selected path or intent • Task decomposition • Confidence flag	*Why agent chose workflow A vs B*
Tool Selection	• Chosen tool • Rejected tools • Decision reason	*Shows right capability was picked*
Tool Arguments	• Parameters sent • Missing values • Validation failures	*Catches 'right tool, wrong input'*
Retrieval / Evidence	• Docs retrieved • Chunk IDs/citations • Relevance score	*Answer grounded in right evidence*
Guardrails / Policy	• Safety result • Policy outcome • Escalation trigger	*Unsafe cases handled correctly*
Action / Handoff	• Approval required? • Action executed? • Side effects	*The blast-radius checkpoint*

Do NOT log raw chain-of-thought

DO log: step ID, tool, params, outputs, evidence, errors, retries, approvals, timestamps

DEMO 2

Follow the Tool Call

Final Answer

= Receipt

Trace

= Security Footage

Trace-level visibility into agent behavior

Control: How You Restrain a Live Agent

Without control, observability is just an autopsy

The Kill Switch Slide

1	**Alert**	Watch but don't stop
2	**Review Mode**	Human approval required
3	**Restricted Mode**	Disable specific tools
4	**Disable Capability**	Turn off specific agent
5	**Stop Agent**	Full shutdown

Monitoring: What You Watch in Production

Metrics that actually matter

Latency
avg 2.3s

Token $
$847/day

Error Rate
0.8%

Tool Failures
3/hr

Quality Score
0.92

Safety Flags
2 today

Escalations
12%

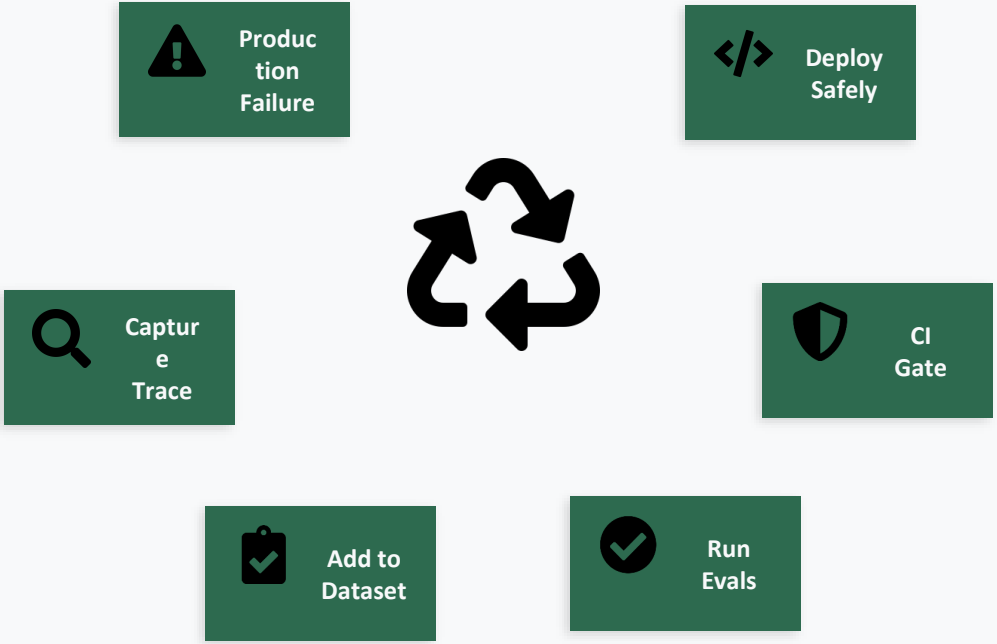
Trending
↑ 5%

Pick 5-7 metrics that matter for your use case. Watch them daily, not weekly.

Feedback Loops

Turn production scars into test cases

Your best eval dataset comes from production embarrassment



What to Do on Monday Morning

Your Starter Kit (Not a Cathedral)

- ✓ Add tracing (OpenTelemetry if you can)
- ✓ Capture tool calls: inputs, outputs, parameters, errors
- ✓ Define 5-10 golden prompts for critical use cases
- ✓ Add ONE regression check to CI
- ✓ Choose one risky action requiring human review
- ✓ Define kill-switch playbook (before you need it)
- ✓ Monitor costs — agents burn tokens like dry brush in August

If it can act, it needs traces, tests, gates, and a kill switch.

Questions?

Brian Haydin
brian.haydin@concurrency.com
linkedin.com/in/brianhaydin